

Day 4: DOM Manipulation & Events

Comprehensive Reference Guide: DOM Tree Architecture, Query Selectors, HTMLCollection vs NodeList, Element Mutation & Event Delegation

Day 4: DOM Manipulation, Elements & Event Handling

Table of Contents

1. Understanding the DOM (Document Object Model)

- What is the DOM Tree?
- Nodes vs. Elements

2. Selecting DOM Elements

- Traditional Selectors (`getElementById` , `getElementsByClassName` , `getElementsByTagName`)
- Modern Selectors (`querySelector` , `querySelectorAll`)
- Attribute Selectors

3. HTMLCollection vs. NodeList (Live vs. Static)

- Deep Dive into Live vs. Static Snapshots
- Comparison Matrix

4. Reading & Modifying DOM Content

- `innerHTML` vs. `textContent` vs. `innerText`
- Form Values (`input.value`)
- Styling via JavaScript (`element.style`)
- Class Manipulation (`classList.add` , `remove` , `toggle`)

5. Creating, Appending & Removing Elements

- `document.createElement()`
- `append()` vs. `appendChild()`
- Removing Elements: `removeChild()` vs. `element.remove()`
- Dynamic Element Generation in Loops

6. Event Handling & Event Listeners

- The `addEventListener` Method
- Common Mouse & Input Events (`click` , `dblclick` , `mouseover`)
- State Toggling with Events

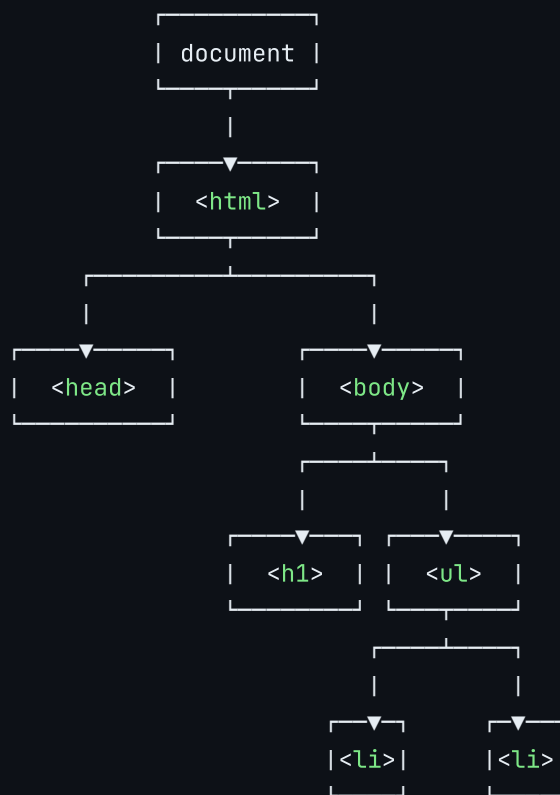
7. Event Propagation & Event Delegation

- Bubbling and Capturing
- Event Delegation (`e.target` vs. `e.currentTarget`)
- Why Delegation is Crucial for Dynamic Lists

8. Day 4 Practice Exercises: In-Depth Solutions

1. Understanding the DOM (Document Object Model)

The **DOM** is a tree-like, object-oriented representation of an HTML document created by the browser. It allows JavaScript to inspect, manipulate, add, or delete HTML elements, attributes, and styles dynamically.



Nodes vs. Elements

- **Node:** Any single point in the DOM tree (Elements, Text nodes, Comments, Document).

- **Element**: A specific type of node representing an HTML tag (e.g., `<div>`, `<p>`, `<li>`).
-

2. Selecting DOM Elements

Traditional Selectors

1. `document.getElementById("id")` :

- Selects a single element matching the unique `id`.
- Returns: A single `Element` object, or `null` if not found.

```
const header = document.getElementById("h1");
console.log(header.tagName); // "H1"
```

2. `document.getElementsByClassName("className")` :

- Selects all elements possessing that class name.
- Returns: A **Live HTMLCollection**.

```
const items = document.getElementsByClassName("Data");
console.log(items.length);
```

3. `document.getElementsByTagName("tagName")` :

- Selects all elements of a given HTML tag (e.g., `'h1'`, `'p'`).
 - Returns: A **Live HTMLCollection**.
-

Modern Selectors

Modern DOM selection relies on CSS selectors:

1. `document.querySelector("cssSelector")` :

- Returns the **first** element that matches the specified CSS selector.
- Returns: `Element` or `null`.

```
const firstCard = document.querySelector(".card");
const mainHeading = document.querySelector("#h1");
const firstParagraph = document.querySelector("div.card > p");
```

2. `document.querySelectorAll("cssSelector")` :

- Returns **all** matching elements.
- Returns: A **Static NodeList**.
- Supports built-in `.forEach()` iterating directly over elements.

```
const allParagraphs = document.querySelectorAll("p");
allParagraphs.forEach((p) => {
  p.style.color = "blue";
});
```

Attribute Selectors

You can query elements by any HTML attribute using CSS bracket notation:

```
// Select input with type="text"
const textInput = document.querySelector('input[type="text"]');
console.log(textInput.value);

// Select button with a specific data attribute
const deleteBtn = document.querySelector('button[data-action="delete"]');
```

3. HTMLCollection vs. NodeList (Live vs. Static)

This is one of the most critical conceptual differences in DOM scripting.

HTMLCollection vs NodeList	
Live HTMLCollection	Static NodeList
Returned by: getElementsByClassName() getElementsByTagName()	Returned by: querySelectorAll()
Automatically updates whenever the DOM tree changes in real-time.	Static snapshot taken at query time. Does NOT update when elements are added or removed from the DOM.
Contains Elements only.	Can contain any Node type (text, comments).
No built-in .forEach(). (Must convert to Array)	Has built-in .forEach() method.

Deep Dive: Why "Live" Matters

```
// Live HTMLCollection demonstration:
const liveItems = document.getElementsByClassName("item");
console.log(liveItems.length); // e.g. 3

// Add a new element to DOM:
const newItem = document.createElement("li");
newItem.className = "item";
document.body.appendChild(newItem);

// Automatically updates without re-querying!
console.log(liveItems.length); // 4

// Static NodeList demonstration:
const staticItems = document.querySelectorAll(".item");
console.log(staticItems.length); // 4

document.body.appendChild(document.createElement("li"));
// Remains unchanged!
console.log(staticItems.length); // Still 4
```

4. Reading & Modifying DOM Content

`innerHTML` vs. `textContent` vs. `innerText`

Property	Parses HTML?	Shows Hidden Text?	Performance	Security Risk
<code>innerHTML</code>	Yes	Yes	Slower (causes re-parsing)	High (XSS attacks)
<code>textContent</code>	No (Raw text)	Yes (including <code><script></code> /styles)	Fastest	Safe
<code>innerText</code>	No (Rendered text)	No (respects CSS <code>display:none</code>)	Slower (triggers reflow)	Safe

```
const box = document.querySelector(".box");

// Overwrites all internal markup:
box.innerHTML = "<strong>Updated content!</strong>";

// Safe replacement of plain text:
box.textContent = "Plain updated text!";
```

Form Values (`input.value`)

For form controls (`<input>` , `<textarea>` , `<select>`), reading or writing the entered value uses the `.value` property, **not** `innerHTML` or `textContent` :

```
const searchInput = document.querySelector('input[name="search"]');
console.log("Current user typed:", searchInput.value);

// Update programmatically:
searchInput.value = "Fresh Apples";
```

Styling via JavaScript (`element.style`)

Styles applied via `.style` are injected as **inline styles** directly onto the HTML element (`style="..."`):

- CSS properties with dashes become **camelCase** in JavaScript:
 - `background-color` $\rightarrow$ `backgroundColor`
 - `font-size` $\rightarrow$ `fontSize`
 - `z-index` $\rightarrow$ `zIndex`

```
const btn = document.querySelector(".btn");
btn.style.color = "white";
btn.style.backgroundColor = "black";
btn.style.padding = "10px 16px";
btn.style.borderRadius = "6px";
```

Class Manipulation (`classList`)

Instead of setting inline styles, modern web development relies on managing CSS classes with `element.classList` :

```
const alertBox = document.querySelector(".alert");

alertBox.classList.add("active", "visible"); // Add one or more classes
alertBox.classList.remove("hidden");         // Remove a class
alertBox.classList.toggle("dark-mode");      // Adds if missing, removes if present
const hasClass = alertBox.classList.contains("active"); // Returns boolean true/false
```

5. Creating, Appending & Removing Elements

`document.createElement()`

Creates a new element in memory, waiting to be inserted into the DOM:

```
const newHeading = document.createElement("h2");
newHeading.textContent = "Welcome to the Fruit Shop!";
newHeading.classList.add("subheading");
```

`append()` vs. `appendChild()`

Feature	<code>appendChild()</code> (Legacy)	<code>append()</code> (Modern)
Arguments	Only 1 Node object	Multiple Nodes AND plain strings
Return Value	Returns the appended Node	Returns <code>undefined</code>
Accepts Text?	No (throws error)	Yes (creates text nodes automatically)

```
const container = document.querySelector(".cart");

// Modern append accepts strings and elements together:
container.append(newHeading, "Enjoy fresh fruits daily!");
```

Removing Elements: `removeChild()` vs. `element.remove()`

1. Traditional `parentNode.removeChild(childNode)` :

Requires a direct reference to both the parent element and the child element:

```
const parent = document.querySelector(".wrapper");
const child = parent.querySelector(".span");

// Must pass an actual DOM node reference, NOT a CSS string!
parent.removeChild(child);
```

2. Modern `element.remove()` :

Can be invoked directly on the element you wish to destroy:

```
const child = document.querySelector(".wrapper .span");
child.remove(); // Clean and direct!
```

Dynamic Element Generation in Loops

When generating repetitive elements (e.g. lists, grids, cards):

```
function generateBoxes(containerId, count) {
  const container = document.getElementById(containerId);

  // DocumentFragment batches DOM updates to minimize browser reflows
  const fragment = document.createDocumentFragment();

  for (let i = 1; i ≤ count; i++) {
    const div = document.createElement("div");
    div.id = `box-${i}`;
    div.className = "box-item";
    div.textContent = `Box #${i}`;
    fragment.appendChild(div);
  }

  container.appendChild(fragment); // Single paint/reflow!
}
```

6. Event Handling & Event Listeners

The `addEventListener` Method

Attaches an event handler function to an element without overwriting existing event handlers:

```
element.addEventListener(eventType, handlerFunction, options);
```



```
const orderBtn = document.querySelector(".btn");

orderBtn.addEventListener("click", function(event) {
  console.log("Order button clicked!");
  this.style.backgroundColor = "#16a34a"; // Green
  this.textContent = "Order Placed!";
});
```

Common Mouse & Input Events

Event Name	Triggered When
click	User clicks with mouse or taps on touchscreen
dblclick	User rapidly double-clicks an element
mouseover	Mouse cursor enters the element or its children
mouseout	Mouse cursor leaves the element
input	Value of an <input> or <textarea> changes live
submit	Form submission is triggered

Hover Effect on Multiple Cards:

```
const cards = document.querySelectorAll(".card");

cards.forEach((card) => {
  card.addEventListener("mouseover", () => {
    card.style.backgroundColor = "#fdf2f8"; // Light pink
  });

  card.addEventListener("mouseout", () => {
    card.style.backgroundColor = ""; // Reset to default
  });
});
```

State Toggling with Events

Toggling states back and forth on user interaction:

```
const heading = document.querySelector("#h1");

heading.addEventListener("dblclick", function() {
  this.textContent = (this.textContent === "Hello!!") ? "Goodbye" : "Hello!!";
});
```

7. Event Propagation & Event Delegation

Bubbling and Capturing

When an event occurs on an element (e.g., clicking a `<li>`), the event doesn't just execute on that element:

- ```
1. Capturing Phase: Window → Document → <body> → →
2. Target Phase: Event executes on
3. Bubbling Phase: → → <body> → Document → Window
```

By default, event listeners run during the **Bubbling Phase** (propagating upwards to parent ancestors).

---

### Event Delegation ( `e.target` vs. `e.currentTarget` )

**Event Delegation** is a design pattern where instead of attaching an event listener to every individual child item, you attach a **single** listener to their common parent.

- `e.target`: The exact inner element that was clicked (e.g., `<li>Apple</li>`).
- `e.currentTarget`: The element currently handling the event (the `<ul>` with the listener).

```
const list = document.querySelector("ul.Data");

// Single listener handles ALL current and future elements!
list.addEventListener("click", function(event) {
 // Check if the clicked target is an LI element
 if (event.target.tagName === "LI") {
 console.log("Clicked item:", event.target.textContent);
 }
});
```

### Why Delegation is Crucial:

1. **Performance**: 1 listener on `<ul>` instead of 1,000 listeners on 1,000 `<li>` tags saves substantial memory.
  2. **Dynamic Elements**: Newly added items dynamically appended later automatically respond to clicks without needing new listeners.
-

## 8. Day 4 Practice Exercises: In-Depth Solutions

---

Refined and bug-free solutions for the exercises in `Day 4/script.js`:

### Q1. Select Element by Two Methods & Log TagName

```
// Method 1: getElementById
const el1 = document.getElementById("h1");
console.log("Method 1 TagName:", el1.tagName); // "H1"

// Method 2: querySelector
const el2 = document.querySelector("#h1");
console.log("Method 2 TagName:", el2.tagName); // "H1"
```

### Q2. `getElementsByClassName` & Count

```
const items = document.getElementsByClassName("Data");
console.log("Elements found:", items.length);
```

### Q3. `querySelectorAll` for all `<p>` and Change Color

```
const paragraphs = document.querySelectorAll("p");
paragraphs.forEach((p) => {
 p.style.color = "blue";
});
```

### Q4. Create `<h2>` with "Welcome!" and Append to Body

```
const h2 = document.createElement("h2");
h2.textContent = "Welcome!";
document.body.appendChild(h2);
```

#### Q5. Select First `.box` and Update `innerHTML`

```
const box = document.querySelector(".box");
if (box) {
 box.innerHTML = "Updated!";
}
```

#### Q6. Button Click Listener

```
const btn = document.querySelector(".btn");
btn.addEventListener("click", function () {
 console.log("Button clicked");
 btn.style.color = "white";
 btn.style.backgroundColor = "black";
});
```

#### Q7. Create `<li>` and Append to `<ul>`

```
const ul = document.querySelector("ul.Data");
const newLi = document.createElement("li");
newLi.textContent = "New Item";
ul.appendChild(newLi);
```

#### Q8. Difference: `HTMLCollection` vs. `NodeList`

- `getElementsByClassName`: Returns a **live HTMLCollection** that dynamically updates when DOM nodes are added/removed. Contains only element nodes; lacks native `.forEach()`.
- `querySelectorAll`: Returns a **static NodeList** representing a snapshot of the DOM at the exact moment of invocation. Does not track future DOM mutations; provides a built-in `.forEach()`.

## Q9. Hover Effect on All `.card` Elements

```
const cards = document.querySelectorAll(".card");
cards.forEach((card) => {
 card.addEventListener("mouseover", function () {
 card.style.backgroundColor = "pink";
 });
 card.addEventListener("mouseout", function () {
 card.style.backgroundColor = "";
 });
});
```

## Q10. Create `<p>`, Add Class via `classList`, and Append

```
const p = document.createElement("p");
p.classList.add("highlight-text");
p.textContent = "Dynamically injected paragraph!";

const container = document.querySelector(".card");
container.appendChild(p);
```

## Q11. Remove Child with `removeChild`

```
const wrapper = document.querySelector(".wrapper");
const childSpan = wrapper.querySelector(".span");

// removeChild requires an actual DOM Node object:
if (wrapper && childSpan) {
 wrapper.removeChild(childSpan);
}
```

## Q12. Double-Click Toggle on `<h1>`

```
const h1 = document.querySelector("#h1");
h1.addEventListener("dblclick", function () {
 h1.textContent = (h1.textContent === "Hello!!") ? "Goodbye" : "Hello!!";
});
```

### Q13. Query by Attribute & Log Value

```
const textInput = document.querySelector('input[type="text"]');
if (textInput) {
 console.log("Input value:", textInput.value);
}
```

### Q14. Function Creating 5 `<div>` Elements with Unique IDs

```
function createFiveBoxes(containerSelector) {
 const parent = document.querySelector(containerSelector);
 for (let i = 1; i ≤ 5; i++) {
 const div = document.createElement("div");
 div.id = `box-${i}`;
 div.className = "generated-box";
 div.textContent = `Box ${i}`;
 parent.appendChild(div);
 }
}
createFiveBoxes(".wrapper");
```

### Q15. Event Delegation on Parent `<ul>`

```
const list = document.querySelector("ul.Data");

list.addEventListener("click", function (e) {
 // Only respond if an was clicked:
 if (e.target.tagName === "LI") {
 console.log("Clicked fruit:", e.target.textContent);
 }
});
```

## Key Takeaways

1. **Selection:** Prefer `querySelector` / `querySelectorAll` for modern, flexible CSS querying.
2. **Collection Types:** Be mindful that `querySelectorAll` creates static snapshots, while `getElementsBy*` tracks live DOM changes.
3. **Safe Content:** Use `textContent` for safety and speed; use `innerHTML` only when inserting actual HTML tags.

4. **Node Removal:** `parentNode.removeChild(node)` requires an Element node parameter, not a CSS selector string.
5. **Event Delegation:** Leverage event bubbling with `e.target` on parent elements to handle multiple children cleanly and dynamically.